📦 CATEGORY 1: Linux Fundamentals — Complete Deep Dive

1.1 Linux Filesystem Hierarchy (FHS)

🔷 What the FHS is

The **Filesystem Hierarchy Standard** defines where everything lives in a Linux system. Unlike Windows where every app picks its own location, Linux follows a tree rooted at `/` with standardized directories. Knowing this hierarchy tells you *where to look* for anything on any Linux system.

🔷 The full directory map

```
/                       root of the entire filesystem tree
├── bin/                essential user binaries (ls, cp, cat) — symlink to
/usr/bin on modern distros
├── sbin/               essential system binaries (fdisk, iptables) —
symlink to /usr/sbin
├── boot/               bootloader files, kernel images, initrd
│   ├── vmlinuz-*       compressed kernel binary
│   ├── initrd.img-*    initial RAM disk
│   └── grub/           GRUB bootloader config
├── dev/                device files (block/char devices)
│   ├── sda             first SATA/SCSI disk
│   ├── nvme0n1         first NVMe disk
│   ├── tty0            first terminal
│   ├── null            null device (discards all writes)
│   ├── zero            zero device (endless zeros)
│   ├── random          random bytes (blocks if entropy low)
│   └── urandom         non-blocking random bytes
├── etc/                system-wide configuration files (text, editable)
│   ├── passwd          user accounts (no passwords)
│   ├── shadow          hashed passwords (root-readable only)
│   ├── group           group definitions
│   ├── fstab           filesystem mount table
│   ├── hosts           local hostname-to-IP mappings
│   ├── resolv.conf     DNS resolver config
│   ├── ssh/            SSH server config
│   └── cron.d/         system crontabs
├── home/               user home directories
├── lib/                essential shared libraries — symlink to /usr/lib on
modern distros
├── media/              mount points for removable media (USB, CDs)
├── mnt/                temporary mount points (manual mounts)
├── opt/                optional/third-party software
├── proc/               virtual filesystem: kernel and process info (RAM
```

```
only, not disk)
│   ├── cpuinfo       CPU information
│   ├── meminfo       memory statistics
│   ├── 1234/         directory for PID 1234
│   │   ├── cmdline   command line that started it
│   │   ├── status    process state, memory usage
│   │   ├── fd/       open file descriptors
│   │   └── maps      memory mappings
│   └── sys/          kernel parameters (also via /sys)
├── root/             root user home (NOT /home/root)
├── run/              runtime data (PIDs, sockets) — cleared at boot
├── srv/              data served by this system (web, FTP)
├── sys/              virtual filesystem: device/driver/kernel info
│   ├── block/        block device info
│   ├── class/        device classes (net, input)
│   └── devices/      device hierarchy
├── tmp/              temporary files (cleared at boot or periodically)
├── usr/              user system resources (read-only in theory)
│   ├── bin/          most user commands live here
│   ├── sbin/         non-essential system binaries
│   ├── lib/          libraries
│   ├── local/        locally compiled/installed software
│   └── share/        architecture-independent data (man pages, docs)
└── var/              variable data (changes frequently)
    ├── log/          system and application logs
    ├── lib/          persistent application state
    ├── cache/        cached data (regeneratable)
    └── spool/        queued data (cron, print, mail)
```

🔷 The virtual filesystems: `/proc` and `/sys`

```
# /proc: process and kernel information (exists only in RAM)
cat /proc/cpuinfo | grep "model name" | head -1
# model name : Intel(R) Xeon(R) CPU E5-2686 v4 @ 2.30GHz

cat /proc/meminfo | grep -E "MemTotal|MemAvailable"
# MemTotal:      16148936 kB
# MemAvailable:  10643916 kB

cat /proc/uptime
# 3456789.12 1234567.89   (seconds since boot, idle seconds)

ls /proc/1234/         # Any running PID
# cmdline  cwd  environ  exe  fd  limits  maps  net  stat  status

cat /proc/1234/cmdline | tr '\0' ' '   # Command that started process
cat /proc/1234/status | grep -E "State|VmRSS|Threads"

# /sys: sysfs — one attribute per file, organized by device class
cat /sys/block/sda/size              # Disk size in 512-byte sectors
```

```
cat /sys/block/sda/queue/scheduler   # I/O scheduler
cat /sys/class/net/eth0/speed        # NIC speed in Mbps
cat /sys/class/net/eth0/operstate    # "up" or "down"

# Writable sysfs entries change kernel behavior live
echo mq-deadline > /sys/block/sda/queue/scheduler
```

🔷 Key directories for daily work

```
# /etc — configuration, all text files, system-wide
grep -r "PasswordAuthentication" /etc/ssh/
diff /etc/hosts /etc/hosts.orig

# /var — variable runtime data
tail -f /var/log/syslog
du -sh /var/lib/docker/
du -sh /var/cache/apt/

# /run — runtime (cleared at boot)
cat /run/nginx.pid
ls /run/systemd/units/

# /tmp vs /var/tmp
# /tmp:     cleared at boot (or by systemd-tmpfiles)
# /var/tmp: survives reboots (persistent temp space)
ls -la / | grep tmp
# drwxrwxrwt 20 root root 4096 /tmp      t = sticky bit
# drwxrwxrwt  8 root root 4096 /var/tmp
```

🔷 Short crisp interview answer

> "The FHS defines a standardized directory tree under `/`. Key directories: `/etc` holds configuration — text files read at startup; `/var` holds variable data like logs and caches; `/proc` and `/sys` are virtual filesystems entirely in RAM that expose kernel and process info — nothing written there is persisted to disk; `/dev` holds device files; `/usr` contains most binaries; `/run` is cleared at every boot and holds runtime state like PID files."

⚠ Gotchas

```
# GOTCHA 1: /proc and /sys files are not real files
du -sh /proc    # Shows 0 — nothing on disk!

# GOTCHA 2: /bin, /sbin, /lib are symlinks on modern distros
ls -la /bin     # lrwxrwxrwx -> usr/bin  (Ubuntu 20.04+, Debian 11+)
```

```
# GOTCHA 3: /tmp is not always cleared at boot
# systemd-tmpfiles controls cleanup — check /usr/lib/tmpfiles.d/tmp.conf

# GOTCHA 4: /dev/random vs /dev/urandom
# /dev/random blocks if entropy pool runs low (common in containers/VMs)
# /dev/urandom never blocks — use this for scripts
```

1.2 File & Directory Operations — `ls`, `cp`, `mv`, `rm`, `find`, `locate`

🔷 `ls` — List directory contents

```
ls -l          # Long format: permissions, links, owner, group, size, date
ls -la         # Include hidden files (. prefix)
ls -lh         # Human-readable sizes (KB, MB, GB)
ls -lt         # Sort by modification time (newest first)
ls -ltr        # Reverse time sort (oldest first — great for log dirs)
ls -lS         # Sort by size (largest first)
ls -R          # Recursive

# Long format breakdown:
ls -la /etc/passwd
# -rw-r--r-- 1 root root 2847 Mar 10 12:00 /etc/passwd
# ──────────── ─ ──── ──── ──── ──────────── ───────────
# permissions  │ user group size    date        name
#              └── hard link count

# File type prefixes:
# -  regular file    d  directory    l  symlink
# b  block device    c  char device  p  pipe    s  socket
```

🔷 `cp`, `mv`, `rm`

```
# cp
cp source.txt dest.txt
cp -r /source/dir/ /dest/dir/        # Recursive directory copy
cp -a /source/dir/ /dest/dir/        # Archive: recursive + preserve all
metadata
cp -p file dest                      # Preserve: timestamps, mode,
ownership
cp -u file dest                      # Update: only copy if source is
newer
cp -i file dest                      # Interactive: prompt before
overwriting
```

```
# Backup before editing (critical habit)
cp /etc/nginx/nginx.conf /etc/nginx/nginx.conf.bak.$(date +%Y%m%d)

# mv
mv oldname.txt newname.txt
mv file.txt /path/to/directory/
mv -i file dest                      # Prompt before overwriting
mv -n file dest                      # Never overwrite existing

# Cross-filesystem move is a copy+delete (slow, uses disk space)
mv /local/bigfile /nfs/bigfile       # Full copy then delete
# Better: rsync + rm for large cross-fs moves

# rm
rm file.txt
rm -r /path/to/directory/            # Recursive
rm -rf /path/                        # Force + recursive (no prompts)
rm -i file                           # Interactive: prompt per file

# Safety: always check before rm
ls -la /path/to/delete/

# Dangerous space-before-wildcard mistake:
rm -rf / tmp/*    # Deletes root! (space between / and tmp)
rm -rf /tmp/*     # Correct — deletes contents of /tmp

# Handle filenames starting with -
rm -- -strangefile

# Safer pattern: find -delete instead of rm -rf
find /var/log -name "*.log" -mtime +30 -print   # Preview first
find /var/log -name "*.log" -mtime +30 -delete  # Then delete
```

🔷 `find` — The workhorse

```
# By name
find /etc -name "*.conf"
find / -name "nginx.conf" 2>/dev/null    # Suppress permission errors
find . -iname "readme*"                  # Case-insensitive

# By type
find /var -type f     # Regular files only
find /var -type d     # Directories only
find /tmp -type l     # Symbolic links

# By size
find / -size +100M         # Larger than 100MB
find /var/log -size +50M   # Large log files
find . -empty              # Empty files and dirs
```

```
# By time
find /var/log -mtime +7    # Modified more than 7 days ago
find /tmp -mtime -1        # Modified in last 24 hours
find /var/log -mmin -60    # Modified in last 60 minutes
find /etc -newer /etc/passwd  # Modified more recently than /etc/passwd

# By permissions
find / -perm 777 2>/dev/null        # World-writable (security check)
find / -perm -4000 2>/dev/null      # SUID files (security audit)
find / -perm -2000 2>/dev/null      # SGID files

# By owner
find /home -user alice
find / -nouser 2>/dev/null          # Files with no valid owner (security
risk!)

# Execute actions
find /var/log -name "*.log" -mtime +30 -exec gzip {} \;    # one cmd per
file
find /var/log -name "*.log" -exec ls -lh {} +              # batch
(faster)
find /var/log -name "*.gz" -mtime +30 -delete              # delete
directly

# Combining conditions
find /var -type f -name "*.log" -size +10M    # AND (implicit)
find /tmp -name "*.tmp" -o -name "*.cache"    # OR
find /etc -name "*.conf" ! -name "default*"   # NOT
find /var -type f \( -name "*.log" -o -name "*.gz" \) -mtime +30  #
grouped

# Production patterns:
# Find large files consuming disk
find / -type f -size +500M 2>/dev/null | xargs ls -lh | sort -k5 -rh

# Security audit: SUID binaries
find / -type f \( -perm -4000 -o -perm -2000 \) -exec ls -la {} \;
2>/dev/null

# Find world-writable directories
find / -type d -perm -o+w 2>/dev/null | grep -v proc | grep -v sys
```

🔷 `locate` — Fast database search

```
locate nginx.conf          # Instant — searches pre-built database
locate -i readme           # Case-insensitive
locate -n 10 "*.conf"      # Limit results
sudo updatedb              # Rebuild database (runs daily via cron/timer)
```

```
# locate vs find:
# locate: instant, may be stale (last updatedb run)
# find:   real-time scan, always current, supports complex criteria
```

🔷 Short crisp interview answer

> "`find` is the workhorse for complex searches — I combine conditions: `find /var/log -type f -name '*.log' -mtime +30 -delete` finds and deletes old log files. For production use, always preview with `-print` before `-delete`. `locate` is instant but uses a stale database — good for 'where does nginx.conf live?' With `rm -rf`, I always run `ls` on the path first and use `find ... -delete` for selective deletion."

⚠ Gotchas

```
# GOTCHA 1: rm -rf with accidental space
rm -rf / tmp/*     # Destroys root!
rm -rf /tmp/*      # Correct

# GOTCHA 2: find -mtime +7 means STRICTLY more than 7 days
# Day 7 itself is excluded

# GOTCHA 3: locate database is stale after new file creation
touch /tmp/newfile && locate newfile   # Not found until next updatedb

# GOTCHA 4: find -exec {} \; vs -exec {} +
find /logs -name "*.log" -exec gzip {} \;   # Slow: one gzip per file
find /logs -name "*.log" -exec gzip {} +    # Fast: batch files together

# GOTCHA 5: cp -r copies symlinks as symlinks (not targets)
cp -L -r /source/ /dest/   # -L = follow and copy link targets
---
---

# 1.3 File Permissions & Ownership — `chmod`, `chown`, `umask`, Sticky
Bit, SUID/SGID

## 🔷 The Unix permission model
```

Every file/directory has three permission sets: Owner (user) | Group | Others

Each set has three bits: r = read = 4 w = write = 2 x = execute = 1

ls -l /etc/passwd

- rw- r-- r-- 1 root root 2847 /etc/passwd | ——— ——— ——— | | | └── others: read only | | └———— group: read only | └———————— owner: read + write └———————————— file type: - = regular

file

```
---

## 🔷 `chmod` — Change permissions

```bash
# Numeric (octal) mode
chmod 755 file     # rwxr-xr-x   owner:rwx  group:r-x  others:r-x
chmod 644 file     # rw-r--r--   owner:rw   group:r--  others:r--
chmod 600 file     # rw-------   owner:rw   group:---  others:---
chmod 700 dir/     # rwx------   owner:rwx  group:---  others:---
chmod 777 file     # rwxrwxrwx   all: full access (DANGEROUS)

# Common patterns:
# 644 → regular files (owner edits, others read)
# 755 → directories and executables
# 600 → private files (SSH keys, passwords)
# 700 → private directories
# 640 → group-readable files

# Symbolic mode — incremental changes
chmod u+x file        # Add execute for owner
chmod g+w file        # Add write for group
chmod o-r file        # Remove read for others
chmod a+r file        # Add read for all
chmod u=rwx,g=rx,o= file  # Set exactly

# Recursive — GOTCHA: sets same on files AND directories
chmod -R 755 /var/www/html/    # Makes all files executable too!

# Better: separate commands for files and dirs
find /var/www/html -type f -exec chmod 644 {} +
find /var/www/html -type d -exec chmod 755 {} +
```

🔷 chown — Change ownership

```
chown alice file.txt                  # Change owner
chown alice:developers file.txt       # Change owner and group
chown :developers file.txt            # Change group only
chgrp developers file.txt             # Same as above

chown -R www-data:www-data /var/www/html/   # Recursive
chown --reference=/etc/hosts /etc/new_config  # Copy from another file
```

🔷 Special permission bits
```

```
# SUID (Set User ID) — bit 4
# On executables: runs as the FILE OWNER, not the calling user
ls -la $(which passwd)
# -rwsr-xr-x 1 root root 59976 /usr/bin/passwd
#     s = SUID (replaces x)
# passwd runs as root even when called by regular user!

chmod u+s /usr/local/bin/myprogram    # Set SUID
chmod 4755 /usr/local/bin/myprogram   # 4 prefix = SUID

# Security audit: find all SUID files
find / -perm -4000 -type f 2>/dev/null | xargs ls -la

# SGID (Set Group ID) — bit 2
# On executables: runs with FILE GROUP, not caller's primary group
# On directories: new files inherit the DIRECTORY GROUP (very useful!)

mkdir /shared/project
chgrp engineering /shared/project
chmod g+s /shared/project        # Set SGID on directory
# Now: all files created inside inherit group "engineering"

ls -la /shared/
# drwxrwsr-x 5 alice engineering /shared/project
#        s = SGID on directory

chmod g+s /shared/project         # Symbolic
chmod 2775 /shared/project        # 2 prefix = SGID

# Sticky Bit — bit 1
# On directories: users can only DELETE their OWN files
# Classic use: /tmp — everyone creates files, only delete your own

ls -la /
# drwxrwxrwt 20 root root 4096 /tmp
#          t = sticky bit (replaces x for others)

chmod +t /shared/dropbox/
chmod 1777 /tmp/          # 1 prefix = sticky

# Summary:
# chmod 4755 = SUID + 755
# chmod 2770 = SGID + 770
# chmod 1777 = sticky + 777 (like /tmp)
```

🔷 `umask` — Default permission mask

```
umask        # Show current umask — typically 0022
```

```
# umask SUBTRACTS permissions from defaults:
# Default files:       666 (rw-rw-rw-)
# Default directories: 777 (rwxrwxrwx)

# With umask 0022:
# File:      666 - 022 = 644 (rw-r--r--)
# Directory: 777 - 022 = 755 (rwxr-xr-x)

# Common umask values:
# 022 → files:644, dirs:755  (standard — world-readable)
# 027 → files:640, dirs:750  (group can read, others blocked)
# 077 → files:600, dirs:700  (private — owner only)

# Set for current session
umask 027

# Set permanently
echo "umask 027" >> ~/.bashrc

# In systemd service unit:
# [Service]
# UMask=0027

# Verify
umask 027
touch testfile && ls -la testfile
# -rw-r----- 1 alice alice 0 testfile  (640)
```

🔷 Short crisp interview answer

> "Linux permissions have three sets (owner, group, others) each with read/write/execute bits. `chmod 755` = owner:rwx group:r-x others:r-x. Special bits: SUID (4) makes an executable run as the file owner — that's how `passwd` writes `/etc/shadow` as a regular user. SGID (2) on a directory makes new files inherit the directory's group — perfect for shared project dirs. Sticky bit (1) on a directory means you can only delete your own files — that's `/tmp`'s 1777. `umask 022` subtracts from defaults: files get 644, directories get 755."

⚠ Gotchas

```
# GOTCHA 1: chmod -R 755 makes files executable too
# Fix: use find+exec to set files and dirs separately

# GOTCHA 2: SUID on scripts is ignored by Linux kernel
# SUID only works on compiled ELF binaries, not shell/python scripts

# GOTCHA 3: Sticky bit on files is obsolete
# Only meaningful on directories in modern Linux
```

```
# GOTCHA 4: umask is not "set permissions to this value"
# It SUBTRACTS from defaults: umask 022 means "remove write for
group/others"

# GOTCHA 5: To delete a file you need write on the PARENT directory, not
the file
rm /dir/file.txt   # Needs write+exec on /dir/, not on file.txt itself
```

1.4 Users, Groups & sudo — `/etc/passwd`, `/etc/shadow`, `useradd`, `sudoers`

🔷 User and group files

```
# /etc/passwd — world-readable, NO passwords (misleading name!)
cat /etc/passwd
# username:x:UID:GID:GECOS:home:shell
# root:x:0:0:root:/root:/bin/bash
# alice:x:1001:1001:Alice Smith,,,:/home/alice:/bin/bash
# www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin

# Field meanings:
# x        → password hash is in /etc/shadow
# UID      → 0=root, 1-999=system accounts, 1000+=human users
# GID      → primary group ID
# GECOS    → full name and info
# shell    → /usr/sbin/nologin or /bin/false = no interactive login

getent passwd alice          # Look up alice (works with LDAP too)
awk -F: '{print $1, $3}' /etc/passwd   # user:UID pairs

# /etc/shadow — root-readable only (mode 640 or 000)
sudo cat /etc/shadow
# alice:$6$salt$hash:19000:0:99999:7:::
# hash formats: $6$=SHA-512, $5$=SHA-256, $1$=MD5, *=locked, !=locked

sudo chage -l alice   # View account aging info

# /etc/group — group definitions
cat /etc/group
# groupname:x:GID:member1,member2,...
# sudo:x:27:alice,bob
# docker:x:999:alice

groups alice          # List all groups for alice
id alice              # uid=1001(alice) gid=1001(alice) groups=...
```

🔷 User management commands

```
# Creating users
sudo useradd -m alice                           # With home directory
sudo useradd -m -s /bin/bash alice             # Home + bash shell
sudo useradd -m -s /bin/bash -G sudo,docker alice  # With additional
groups
sudo adduser alice    # Debian/Ubuntu: interactive, friendlier

# Setting/changing passwords
sudo passwd alice     # Set password (root sets any user's)
sudo passwd -l alice  # Lock account (prepend ! to hash)
sudo passwd -u alice  # Unlock account
sudo passwd -e alice  # Expire password (force change next login)

# Modifying existing users
sudo usermod -aG sudo alice        # Add to group (KEEP EXISTING with -a!)
sudo usermod -aG docker,wheel alice
sudo usermod -s /bin/bash alice    # Change shell
sudo usermod -L alice              # Lock account
sudo usermod -U alice              # Unlock account

# THE CRITICAL GOTCHA:
sudo usermod -G docker alice       # REPLACES all groups - alice only in
docker now!
sudo usermod -aG docker alice      # APPENDS docker, keeps all existing
groups

# Deleting users
sudo userdel alice            # Keep home directory
sudo userdel -r alice         # Delete user AND home directory

# Group management
sudo groupadd engineering
sudo gpasswd -a alice engineering    # Add user to group
sudo gpasswd -d alice engineering    # Remove user from group

# Service accounts - for applications, not humans
sudo useradd -r -s /usr/sbin/nologin -d /var/lib/myapp myapp
# -r: system account (UID < 1000, no home by default)
# -s /usr/sbin/nologin: cannot login interactively
```

🔷 Short crisp interview answer

> "`/etc/passwd` is world-readable and holds user info (UID, GID, home, shell) — the `x` means look at `/etc/shadow` for the password hash. `/etc/shadow` is root-only and holds hashed passwords. `useradd -m -s /bin/bash -G sudo alice` creates a user with home directory, bash shell, and sudo access. Critical gotcha: `usermod -G` without `-a` REPLACES all groups — always use `usermod -aG groupname user` to add without removing existing memberships."

⚠ Gotchas

```
# GOTCHA 1: usermod -G replaces groups (most common mistake!)
sudo usermod -G docker alice   # alice is now ONLY in docker — lost sudo!
sudo usermod -aG docker alice  # -a = append — correct!

# GOTCHA 2: New group membership requires re-login
sudo usermod -aG docker alice
# alice must log out and back in (or: newgrp docker in current session)

# GOTCHA 3: /etc/passwd doesn't have passwords
# World-readable — never put secrets in GECOS field

# GOTCHA 4: userdel without -r leaves orphaned home directory
sudo userdel alice     # /home/alice still exists — potential security
issue
find / -nouser 2>/dev/null  # Find files with no valid owner
```

1.5 Text Viewing & Editing — `cat`, `less`, `head`, `tail`, `vim`, `nano`

🔷 `cat`, `less`, `head`, `tail`

```
# cat — display and concatenate
cat file.txt
cat -n file.txt            # Show line numbers
cat -A file.txt            # Show non-printing chars (^M$ = Windows CRLF)
cat file1.txt file2.txt    # Concatenate

# Create file with heredoc
cat > /tmp/config.txt << 'EOF'
server=localhost
port=5432
EOF

# less — pager (correct way to view large files)
less /var/log/syslog

# less navigation:
# SPACE/f   → page down         b      → page up
# g         → top               G      → bottom
# /pattern  → search forward    ?pattern → search backward
# n         → next match        N      → previous match
# F         → follow mode (like tail -f)
# q         → quit
```

```
less -N /etc/nginx/nginx.conf    # With line numbers
less +G /var/log/syslog          # Open at end
less +F /var/log/syslog          # Open in follow mode

# head — first N lines
head file.txt             # First 10 lines (default)
head -n 20 file.txt       # First 20 lines
head -n -5 file.txt       # All except last 5 lines

# tail — last N lines
tail file.txt             # Last 10 lines (default)
tail -n 50 file.txt       # Last 50 lines
tail -n +5 file.txt       # From line 5 onwards

# tail -f — follow live log files
tail -f /var/log/syslog
tail -F /var/log/syslog   # -F = follow by NAME (survives log rotation!)
tail -f /var/log/nginx/access.log | grep --line-buffered "ERROR"

# Multiple files
tail -f /var/log/nginx/access.log /var/log/nginx/error.log
```

🔷 `vim` — The modal editor

```
# Modes:
# Normal   (default) → navigate and command
# Insert             → type text
# Visual             → select text
# Command            → ex commands (:q, :w, etc.)

# Mode switching:
# Normal → Insert: i (before cursor), a (after), o (new line below)
# Insert → Normal: Esc
# Normal → Command: : (colon)

# Essential Normal mode:
# Navigation:  h j k l  (left/down/up/right)
#              w b       (forward/backward word)
#              0 $       (start/end of line)
#              gg G      (top/bottom of file)
#              :42       (go to line 42)
#
# Editing:     dd        (delete/cut line)
#              yy        (yank/copy line)
#              p         (paste after)
#              u         (undo)
#              .         (repeat last command)
#
# Search:      /pattern  (search forward)
```

```
#               n N       (next/previous match)
#               :%s/old/new/g  (replace all)

# Command mode essentials:
:w            # Save
:q            # Quit (fails if unsaved)
:wq           # Save and quit
:q!           # Quit WITHOUT saving (when stuck!)
:set paste    # Paste mode (prevents auto-indent breaking pasted code)
:set number   # Show line numbers

# SURVIVAL MINIMUM — know these cold:
# i      → insert mode
# Esc    → back to normal mode
# :wq    → save and exit
# :q!    → exit WITHOUT saving
# /pattern → search
```

🔷 nano — Beginner-friendly editor

```
nano file.txt      # Open file — shortcuts shown at bottom

# Key shortcuts (^ = Ctrl, M = Alt):
# ^O  → Write Out (save)
# ^X  → Exit
# ^W  → Search
# ^K  → Cut line
# ^U  → Uncut/paste
# ^_  → Go to line number
# M-U → Undo

nano -l file.txt   # Show line numbers
nano +42 file.txt  # Open at line 42
```

🔷 Short crisp interview answer

> "`less` is the right tool for large files — unlike `cat` it doesn't load everything at once. `/pattern` searches forward, `G` goes to end, `F` follows live. For live log monitoring I use `tail -F` (capital F) which follows by filename and survives log rotation. In vim: `i` enters insert mode, `Esc` returns to normal, `:wq` saves and exits, `:q!` exits without saving. Knowing vim basics is mandatory — it's the only editor guaranteed to be on any server."

⚠ Gotchas

```
# GOTCHA 1: cat on large files floods terminal
cat /var/log/syslog    # Don't! Use less

# GOTCHA 2: tail -f vs tail -F
tail -f /var/log/app.log   # Follows by fd — stops after log rotation!
tail -F /var/log/app.log   # Follows by name — survives rotation (use
this)

# GOTCHA 3: Pasting into vim without :set paste
# Multi-line paste triggers auto-indent — each line gets increasingly
indented
# Fix: :set paste before pasting, :set nopaste after

# GOTCHA 4: Stuck in vim
# Esc multiple times → :q! → Enter — this always exits

# GOTCHA 5: head -n -5 (GNU-specific)
head -n -5 file.txt    # Prints all except last 5 (GNU head only)
```

1.6 Hard Links vs Soft Links — `ln`, Inodes, How They Differ

🔷 What inodes are

```
Every file has three components:
  1. INODE — metadata: permissions, owner, timestamps, size, block
pointers
             (NOT the filename — that's in the directory)
  2. DIRECTORY ENTRY — maps filename → inode number
  3. DATA BLOCKS — the actual file contents

           Directory
   ┌──────────────────────────┐
   │  "file.txt"  → inode 12345  │
   └──────────────────────────┘
                  │
                  ▼
          Inode 12345
   ┌──────────────────────────┐
   │  permissions: 644        │
   │  owner: alice            │
   │  size: 4096 bytes        │
   │  hard link count: 1      │
   │  data block ptr: [123]   │
   └──────────────────────────┘
                  │
                  ▼
```

```
            Block 123
          [actual data]
```

🔷 Hard links

```
# Hard link: another directory entry pointing to the SAME inode
ln original.txt hardlink.txt

ls -lai
# 12345 -rw-r--r-- 2 alice alice 4096 original.txt
# 12345 -rw-r--r-- 2 alice alice 4096 hardlink.txt
#  ↑                  ↑
# same inode!    link count = 2 (two names for same file)

# Properties:
# - Same inode: same permissions, owner, timestamps
# - Both names are equally valid — no "original" vs "link"
# - Deleting one only decrements link count (data survives)
# - Data freed only when link count reaches 0 AND no open fd
# - CANNOT span filesystems (inodes are per-filesystem)
# - CANNOT link directories (would create loops)

rm original.txt    # Link count 2 → 1 (hardlink.txt still works)
rm hardlink.txt    # Link count 1 → 0 (data blocks freed)

# Directories always have at least 2 hard links:
ls -la /etc | head -2
# drwxr-xr-x 165 root root /etc
# ↑ link count 2 = from parent + from /etc/. itself
# Each subdirectory adds 1 more (its .. entry)
```

🔷 Soft (symbolic) links

```
# Soft link: a file containing a PATH to another file
ln -s /original/file.txt symlink.txt
ln -s /path/to/directory/ shortcut/

ls -la symlink.txt
# lrwxrwxrwx 1 alice alice 17 Mar 10 symlink.txt -> /original/file.txt
# ↑ l = symlink type, permissions always lrwxrwxrwx (irrelevant)

# Properties:
# - Different inode from target
# - Contains a path string (not an inode reference)
# - Can span filesystems
# - Can link to directories
# - If target deleted: symlink becomes DANGLING (broken)
```

```
# - Permissions always lrwxrwxrwx (target's permissions used)

# Dangling symlink:
rm /original/file.txt
cat symlink.txt          # Error: No such file or directory
file symlink.txt         # "broken symbolic link to /original/file.txt"

# Find broken symlinks:
find /path -xtype l      # GNU find: finds broken symlinks

# Absolute vs relative symlinks:
ln -s /usr/local/bin/python3.11 /usr/local/bin/python3    # Absolute -
always works
cd /usr/local/bin && ln -s python3.11 python3             # Relative -
moves with link

# Force overwrite existing symlink
ln -sf /new/target existing_symlink    # -f = force

# Common patterns:
# nginx sites-enabled
ln -s /etc/nginx/sites-available/myapp /etc/nginx/sites-enabled/myapp

# Version management
ls -la /usr/bin/python*
# lrwxrwxrwx python  -> python3
# lrwxrwxrwx python3 -> python3.11
# -rwxr-xr-x python3.11
```

🔷 Comparison table

```
Feature              | Hard Link          | Soft (Symbolic) Link
---------------------|--------------------|---------------------------
What it is           | Directory entry    | File containing a path
Inode                | Same as original   | Own inode
Cross-filesystem     | NO                 | YES
Link to directory    | NO (root only)     | YES
If target deleted    | File survives      | Dangling link (broken)
ls -la               | Same inode number  | Shows -> target
find -type           | -type f            | -type l
```

🔷 Short crisp interview answer

> "A hard link is a second directory entry pointing to the same inode — same file, different name. Data isn't deleted until ALL hard links are removed AND no process has the file open. A soft link is a file containing a path — it can cross filesystems and link directories, but becomes dangling if the target is deleted. Hard links can't cross filesystems because inodes are per-filesystem. I use soft links for

> nginx `sites-enabled`, version management (`python3 -> python3.11`), and cross-filesystem references."

⚠ Gotchas

```
# GOTCHA 1: rm on a hard link doesn't delete data
rm file.txt      # If hardlink.txt exists, data survives

# GOTCHA 2: Relative symlinks depend on link's location
# Moving the link without the target breaks it

# GOTCHA 3: cp behavior with symlinks
cp symlink.txt copy.txt    # Copies file CONTENT (follows link)
cp -P symlink.txt copy.txt # Copies the symlink ITSELF

# GOTCHA 4: Updating a symlink requires -f
ln -s /new/target existing_symlink    # Error: file exists
ln -sf /new/target existing_symlink   # Force: works
```

1.7 Disk & Filesystem Management

🔷 Quick reference

```
# How full are my filesystems?
df -h                   # Human-readable filesystem usage
df -i                   # Inode usage (can be "full" even when data space
available)

# What's using disk space?
du -sh /var/*/          # Size of each subdirectory
du -sh /* 2>/dev/null | sort -rh | head   # Top disk consumers

# What block devices exist?
lsblk                   # Tree view of disks and partitions
lsblk -f                # Include filesystem type and UUID

# Mount a filesystem
sudo mount /dev/sdb1 /mnt/data
sudo mount -o ro /dev/sdb1 /mnt/data    # Read-only
sudo umount /mnt/data

# Persistent mounts (/etc/fstab — always use UUID!)
UUID=$(blkid -s UUID -o value /dev/sdb1)
echo "UUID=$UUID /data xfs defaults,noatime 0 2" | sudo tee -a /etc/fstab

# Partition and format
```

```
sudo fdisk -l /dev/sdb                # List partitions
sudo parted /dev/sdb print
sudo parted /dev/sdb mklabel gpt
sudo parted /dev/sdb mkpart primary xfs 1MiB 100%
sudo mkfs.xfs /dev/sdb1
```

See Category 6 (Storage & I/O) for full deep-dive on these tools.

1.8 File Descriptor Internals — stdin/stdout/stderr, `/proc/PID/fd`

🔷 What file descriptors are

```
A file descriptor (fd) is an integer the kernel uses to track open
resources (files, sockets, pipes, devices) per process.

Standard file descriptors (inherited at process creation):
  0 → stdin  (standard input)   — keyboard by default
  1 → stdout (standard output)  — terminal by default
  2 → stderr (standard error)   — terminal by default

When process opens a file:
  open("/etc/hosts")     → returns fd 3
  open("/var/log/app")   → returns fd 4
```

🔷 Redirection mechanics

```
# Basic redirection
command > file          # stdout (fd 1) to file (truncate)
command >> file         # stdout to file (append)
command < file          # stdin (fd 0) from file
command 2> file         # stderr (fd 2) to file
command 2>&1            # stderr to wherever stdout goes
command > file 2>&1     # Both stdout and stderr to file
command &> file         # Bash shorthand: both to file

# ORDER MATTERS for 2>&1:
command > file 2>&1     # CORRECT: stdout→file, stderr→file (same)
command 2>&1 > file     # WRONG: stderr→terminal (original stdout),
stdout→file

# /dev/null
command > /dev/null 2>&1   # Discard everything
command 2>/dev/null         # Discard only errors
```

```
# Pipes
ls -la | grep ".conf"
# ls stdout → pipe → grep stdin

command 2>&1 | grep pattern     # Pipe both stdout and stderr

# View open fds for a process
ls -la /proc/1234/fd/
# lrwx------ 0 -> /dev/pts/0       (stdin: terminal)
# lrwx------ 1 -> /dev/pts/0       (stdout: terminal)
# lrwx------ 2 -> /dev/pts/0       (stderr: terminal)
# lrwx------ 3 -> /var/log/app.log (open log file)
# lrwx------ 4 -> socket:[12345]   (TCP socket)

lsof -p 1234                        # All open files for process
ls /proc/1234/fd | wc -l            # Count open fds

# File descriptor limits
ulimit -n                           # Soft limit: max open files
ulimit -Hn                          # Hard limit
cat /proc/1234/limits | grep "open files"

# Increase limits permanently (/etc/security/limits.conf):
# nginx   soft  nofile  65536
# nginx   hard  nofile  65536

# In systemd service: LimitNOFILE=65536

# Redirect all script output to log (common pattern)
exec > /var/log/myscript.log 2>&1
echo "everything goes to log now"

# Here-strings and process substitution
grep "pattern" <<< "string to search"
diff <(sort file1.txt) <(sort file2.txt)   # Process substitution

# Deleted files held open by fd
rm /var/log/app.log    # Deleted, but if process has fd open:
lsof +L1               # Shows files with link count 0 — still using disk
space!
```

🔷 Short crisp interview answer

> "File descriptors are integers the kernel uses to track open resources per process. 0=stdin, 1=stdout, 2=stderr. `command > file 2>&1` — order matters: first redirect stdout to file, then redirect stderr to wherever stdout now points (the file). Reversed (`2>&1 > file`), stderr goes to the original stdout (terminal). I use `/proc/<PID>/fd/` to see all open files for a process. File descriptor limits are critical for web servers and databases — nginx needs `LimitNOFILE=65536` in its systemd unit."

⚠ Gotchas

```
# GOTCHA 1: Order of 2>&1 relative to >
command > file 2>&1    # Both to file (correct)
command 2>&1 > file    # stderr to terminal, stdout to file (wrong!)

# GOTCHA 2: stderr bypasses pipes
ls /nonexistent | grep total   # stderr goes to terminal, not through
pipe!
ls /nonexistent 2>&1 | grep total  # Now stderr goes through pipe

# GOTCHA 3: Pipe subshell loses variables
echo "hello" | read var    # var not set in parent shell!
read var <<< "hello"       # Works: here-string, no subshell

# GOTCHA 4: Deleted files held open consume disk space
# Only freed when ALL fds are closed — restart process or truncate:
# > /proc/1234/fd/3   (truncate open fd directly)
```

1.9 `/proc` and `/sys` — Virtual Filesystems, Kernel Interfaces

🔷 `/proc` — Process and kernel information

```
# /proc is entirely in RAM — kernel generates content on read

# Per-process directories
ls /proc/1234/
# cmdline  cwd  environ  exe  fd/  limits  maps  net/  ns/  status

cat /proc/1234/cmdline | tr '\0' '\n'    # Command + args (null-separated)
cat /proc/1234/status                    # Human-readable process info
cat /proc/1234/io                        # I/O statistics
cat /proc/1234/maps                      # Memory map
cat /proc/1234/limits                    # Applied ulimits
ls -la /proc/1234/fd/                   # Open file descriptors
ls -la /proc/1234/exe                   # Path to the executable

# System-wide files
cat /proc/cpuinfo                       # CPU details
cat /proc/meminfo                       # Memory statistics
cat /proc/uptime                        # Seconds since boot
cat /proc/loadavg                       # 1/5/15min load averages
cat /proc/version                       # Kernel version
cat /proc/filesystems                   # Supported fs types
cat /proc/mounts                        # Current mounts (kernel's view)
```

```
cat /proc/net/tcp                       # TCP connection table
cat /proc/diskstats                     # Disk I/O stats (used by iostat)
cat /proc/interrupts                    # CPU interrupt counts

# /proc/sys — kernel parameters (same as sysctl)
cat /proc/sys/vm/swappiness
cat /proc/sys/net/core/somaxconn
cat /proc/sys/fs/file-max
cat /proc/sys/net/ipv4/ip_forward

# Write to change kernel parameter
echo 1 > /proc/sys/net/ipv4/ip_forward    # Enable IP forwarding
# Equivalent to: sysctl -w net.ipv4.ip_forward=1

sysctl vm.swappiness              # Read
sysctl -w vm.swappiness=10        # Write (temporary)
sysctl -p                         # Apply /etc/sysctl.conf (persistent)
```

🔷 `/sys` — sysfs: device and driver information

```
# /sys is more structured than /proc — one attribute per file

# Block devices
ls /sys/block/
# dm-0  loop0  nvme0n1  sda  sdb

cat /sys/block/sda/size              # Disk size in 512-byte sectors
cat /sys/block/sda/removable         # 0=fixed, 1=removable
cat /sys/block/sda/queue/scheduler   # I/O scheduler
cat /sys/block/sda/queue/rotational  # 0=SSD, 1=HDD

echo mq-deadline > /sys/block/sda/queue/scheduler    # Change scheduler

# Network interfaces
cat /sys/class/net/eth0/speed        # NIC speed in Mbps
cat /sys/class/net/eth0/operstate    # "up" or "down"
cat /sys/class/net/eth0/address      # MAC address
cat /sys/class/net/eth0/statistics/rx_bytes

# CPU and power
cat /sys/devices/system/cpu/cpu0/cpufreq/scaling_governor
cat /sys/class/thermal/thermal_zone0/temp   # CPU temp in millidegrees
cat /sys/class/power_supply/BAT0/capacity   # Battery %

# Kernel modules
ls /sys/module/                      # All loaded modules
cat /sys/module/tcp_bbr/parameters/  # Module parameters
```

🔷 Short crisp interview answer

> "`/proc` and `/sys` are virtual filesystems — nothing on disk, kernel generates content dynamically on read. `/proc` focuses on process info (each PID gets a directory with cmdline, fds, memory maps, limits) and system stats. `/sys` is more structured, focusing on devices and drivers — one attribute per file. `/proc/sys/` and `sysctl` are two interfaces to the same kernel parameters. I read `/proc/PID/fd/` for open file descriptors, `/proc/net/tcp` for TCP connections, and write to `/proc/sys/` or use `sysctl -w` for runtime kernel tuning."

1.10 `systemd` & Service Management — Units, Targets, `journalctl`, `systemctl`

🔷 What systemd is

systemd is PID 1 — the first process started by the kernel, parent of all others. It manages services, mounts, timers, sockets, and system state.

```
Unit types:
  .service  → a daemon/process to manage
  .timer    → scheduled jobs (cron replacement)
  .mount    → filesystem mount points
  .socket   → socket activation (start service on first connection)
  .target   → group of units (like runlevels)
  .path     → monitor filesystem paths
  .slice    → cgroup resource management
```

🔷 `systemctl` — Service and unit management

```
# Service lifecycle
sudo systemctl start nginx
sudo systemctl stop nginx
sudo systemctl restart nginx
sudo systemctl reload nginx           # Reload config without restart (if
supported)
sudo systemctl try-restart nginx      # Restart only if currently running

# Enable/disable (boot behavior)
sudo systemctl enable nginx           # Start at boot
sudo systemctl disable nginx          # Don't start at boot
sudo systemctl enable --now nginx     # Enable AND start immediately (best
practice)
sudo systemctl disable --now nginx    # Disable AND stop immediately
sudo systemctl mask nginx             # Prevent ANY starting (even manual)
sudo systemctl unmask nginx           # Remove mask
```

```
# Status and inspection
systemctl status nginx                # Full status with recent logs
systemctl is-active nginx             # "active" or "inactive"
systemctl is-enabled nginx            # "enabled" or "disabled"
systemctl is-failed nginx             # "failed" or "active"

# List units
systemctl list-units --type=service        # All active services
systemctl list-units --state=failed        # Failed units
systemctl list-unit-files                  # All installed unit files

# System state
systemctl get-default                      # Default target
sudo systemctl set-default multi-user.target
sudo systemctl poweroff
sudo systemctl reboot

# CRITICAL: after editing unit files
sudo systemctl daemon-reload               # Always required!
```

🔷 Unit files — service definition

```
# Unit file locations (priority, highest first):
# /etc/systemd/system/          local overrides (use this)
# /run/systemd/system/          runtime (cleared at boot)
# /lib/systemd/system/          package-installed (don't edit directly)

systemctl cat nginx              # Show effective unit file (with
overrides)

# Example service unit: /etc/systemd/system/myapp.service
[Unit]
Description=My Application Server
After=network.target postgresql.service    # Start after these
Requires=postgresql.service               # Must succeed for us to start
Wants=redis.service                       # Optional dependency

[Service]
Type=simple            # Process stays in foreground
# Type=forking         # Old-style daemon that forks
# Type=oneshot         # Run and exit (scripts/tasks)
# Type=notify          # Process sends ready notification

User=myapp
Group=myapp
WorkingDirectory=/opt/myapp
ExecStart=/usr/local/bin/myapp --config /etc/myapp/config.yaml
ExecReload=/bin/kill -HUP $MAINPID
```

```
Restart=on-failure     # Restart on non-zero exit
RestartSec=5s
StartLimitBurst=5
StartLimitInterval=60s

# Security hardening
NoNewPrivileges=yes
PrivateTmp=yes

# Resource limits
LimitNOFILE=65536
MemoryLimit=1G
CPUQuota=50%

StandardOutput=journal
StandardError=journal
EnvironmentFile=/etc/myapp/env

[Install]
WantedBy=multi-user.target

# Override without editing original (ALWAYS do this, never edit /lib/...)
sudo systemctl edit nginx
# Creates: /etc/systemd/system/nginx.service.d/override.conf
# Only specify what you want to override:
[Service]
LimitNOFILE=65536

sudo systemctl daemon-reload    # Apply changes
```

🔷 `journalctl` — Querying structured logs

```
# Basic viewing
journalctl                          # All logs (oldest first)
journalctl -r                       # Reverse (newest first)
journalctl -f                       # Follow (like tail -f)
journalctl -n 50                    # Last 50 lines
journalctl -e                       # Jump to end

# Filter by unit
journalctl -u nginx                 # nginx logs
journalctl -u nginx -f              # Follow nginx
journalctl -u nginx -u postgresql   # Multiple services

# Filter by time
journalctl --since "1 hour ago"
journalctl --since "2026-03-10 14:00:00"
journalctl --since today
journalctl --since yesterday
```

```
# Filter by priority
journalctl -p err                   # Error and above
journalctl -p warning               # Warning and above
# Priorities: emerg alert crit err warning notice info debug

# Filter by boot
journalctl -b                       # Current boot
journalctl -b -1                    # Previous boot
journalctl --list-boots             # All recorded boots

# Kernel messages
journalctl -k                       # Like dmesg
journalctl -k -b -1                 # Previous boot kernel messages

# Output formats
journalctl -u nginx -o json | python3 -m json.tool
journalctl -u nginx -o cat          # Just the message, no metadata

# Disk usage and cleanup
journalctl --disk-usage
sudo journalctl --vacuum-size=500M
sudo journalctl --vacuum-time=7d

# Common patterns
journalctl -u myapp -n 100 --no-pager    # Why did service fail?
journalctl -p err --since "1 hour ago"  # Recent errors
journalctl -k | grep -i "oom\|killed"   # OOM events
```

🔷 systemd targets

```
# Traditional runlevels → systemd targets
# runlevel 0 → poweroff.target
# runlevel 1 → rescue.target (single user)
# runlevel 3 → multi-user.target (headless server)
# runlevel 5 → graphical.target (GUI)
# runlevel 6 → reboot.target

systemctl list-dependencies multi-user.target
systemctl list-dependencies --reverse nginx.service  # What depends on
nginx?
```

🔷 Short crisp interview answer

> "systemd is PID 1 — it manages all services. `systemctl enable --now nginx` both enables at boot AND starts immediately. After editing any unit file, `systemctl daemon-reload` is mandatory — without it, systemd uses the old version. `journalctl -u nginx -f` follows nginx logs; `journalctl -u nginx --since '1 hour ago' -p err` shows recent errors. Never edit

> `/lib/systemd/system/` files — use `systemctl edit` to create override files in `/etc/systemd/system/`."

⚠ Gotchas

```
# GOTCHA 1: daemon-reload is ALWAYS required after editing unit files
sudo nano /etc/systemd/system/myapp.service
# Without daemon-reload: systemctl restart uses the OLD unit!
sudo systemctl daemon-reload && sudo systemctl restart myapp

# GOTCHA 2: enable doesn't start, start doesn't persist
sudo systemctl start nginx    # Running now, lost after reboot
sudo systemctl enable nginx   # Starts after reboot, not started now
sudo systemctl enable --now nginx  # Does both

# GOTCHA 3: mask is stronger than disable
sudo systemctl disable nginx  # Won't start at boot, but manual start
works
sudo systemctl mask nginx     # Prevents ALL starting including manual

# GOTCHA 4: Restart=always vs Restart=on-failure
# Restart=always: restarts even on clean exit (exit 0)
# Restart=on-failure: only on non-zero exit (usually what you want)

# GOTCHA 5: Type=forking requires PIDFile=
# Type=simple is better for modern applications
```

1.11 Linux Boot Process — BIOS/UEFI → GRUB → Kernel → initrd → systemd

🔷 The full boot sequence

```
Step 1: POWER ON
  │
  ▼
Step 2: BIOS/UEFI firmware
  BIOS: legacy, 16-bit, reads 512-byte MBR from disk sector 0
  UEFI: modern, 64-bit, reads EFI application from EFI System Partition
  Actions: POST (Power-On Self-Test), initialize hardware, find bootable
device
  │
  ▼
Step 3: Bootloader (GRUB2)
  Files: /boot/grub/grub.cfg  (generated, don't edit directly)
         /etc/default/grub    (user-editable settings)
```

```
    Actions: Show boot menu, load kernel + initramfs into RAM,
             pass kernel parameters, hand off to kernel
    │
    ▼
  Step 4: Kernel initialization
    /boot/vmlinuz-5.15.0-generic (compressed kernel binary)
    Actions: Decompress itself, initialize CPU/memory, mount initramfs as
             temporary root, run /init from initramfs
    │
    ▼
  Step 5: initrd / initramfs
    Small temporary filesystem in RAM
    Purpose: load drivers needed to mount the REAL root filesystem
             (disk controllers, RAID/LVM, LUKS encryption)
    Actions: Load essential modules, set up LVM/RAID/LUKS,
             mount real root filesystem, hand off to /sbin/init
    │
    ▼
  Step 6: systemd (PID 1)
    Actions: Read unit files, mount filesystems (/etc/fstab),
             start services in parallel (dependency order),
             reach default.target — system is ready
```

🔷 GRUB configuration

```
# User-editable GRUB settings
cat /etc/default/grub
# GRUB_DEFAULT=0                          default menu entry
# GRUB_TIMEOUT=5                          seconds to show menu
# GRUB_CMDLINE_LINUX_DEFAULT="quiet splash"  kernel params for normal boot
# GRUB_CMDLINE_LINUX=""                   kernel params for ALL boots

# Common kernel parameters:
# quiet         → suppress boot messages
# splash        → show splash screen
# nomodeset     → disable kernel mode setting (fix GPU issues)
# single        → boot to single-user mode
# rd.break      → break before pivoting to real root (password recovery)
# systemd.unit=rescue.target  → boot to rescue mode
# mem=4G        → limit RAM
# acpi=off      → disable ACPI

# After editing /etc/default/grub:
sudo update-grub                          # Debian/Ubuntu
sudo grub2-mkconfig -o /boot/grub2/grub.cfg  # RHEL/CentOS

# Password recovery procedure:
# 1. Boot → Esc/Shift to get GRUB menu
# 2. Select kernel → press 'e' to edit
# 3. Find line starting with 'linux' or 'linuxefi'
```

```
# 4. Append: rd.break  (before 'ro')
# 5. Ctrl+X to boot
# 6. In emergency shell:
mount -o remount,rw /sysroot
chroot /sysroot
passwd root
touch /.autorelabel    # For SELinux systems
exit && reboot
```

🔷 Kernel and initramfs

```
# Kernel files
ls /boot/
# vmlinuz-5.15.0-91-generic      compressed kernel
# initrd.img-5.15.0-91-generic   initramfs
# System.map-5.15.0-91-generic   kernel symbol table

uname -r     # 5.15.0-91-generic (current kernel)
uname -a     # Full info: version, hostname, arch, build date

# Rebuild initramfs (after adding drivers or changing hooks)
sudo update-initramfs -u              # Debian/Ubuntu
sudo dracut --force                   # RHEL/CentOS

# Inspect initramfs contents
lsinitramfs /boot/initrd.img-$(uname -r)   # Debian/Ubuntu
lsinitrd /boot/initrd $(uname -r)          # RHEL/CentOS
```

🔷 Short crisp interview answer

> "The Linux boot sequence: BIOS/UEFI initializes hardware and loads GRUB2. GRUB loads the kernel (`vmlinuz`) and initramfs into RAM. The kernel decompresses itself, initializes CPU and memory, mounts initramfs as a temporary root, and runs its `/init`. initramfs loads the drivers needed to access the real root filesystem (RAID, LVM, LUKS), mounts it, then hands off to systemd as PID 1. systemd reads unit files, starts services in parallel based on dependencies, and reaches the default target. I've used `rd.break` in GRUB to break before the root pivot for password recovery on locked systems."

1.12 Kernel Modules — `lsmod`, `modprobe`, `insmod`, `rmmod`

🔷 What kernel modules are

```
The Linux kernel is monolithic with modular extensions.
Modules (.ko files) can be loaded/unloaded at runtime without rebooting.

Common modules:
  ext4         → ext4 filesystem support
  xfs          → XFS filesystem support
  overlay      → overlayfs (needed for Docker)
  br_netfilter → bridge netfilter (needed for Kubernetes)
  tcp_bbr      → BBR congestion control
  8021q        → VLAN support
```

🔷 Module management

```
# List loaded modules
lsmod
# Module           Size    Used by
# overlay        147456    0
# br_netfilter    32768    0
# ip_tables       32768    4 iptable_filter,iptable_mangle
# Column 3 "Used by": modules that depend on this one

# Show module information
modinfo tcp_bbr
# filename:     /lib/modules/5.15.0/kernel/net/ipv4/tcp_bbr.ko
# description:  TCP BBR congestion control
# parm:         bbr_bw_rtts:u int  (module parameters)

# Load modules
sudo modprobe tcp_bbr          # Smart: resolves dependencies
automatically
sudo modprobe overlay
sudo modprobe br_netfilter

# modprobe vs insmod:
# modprobe: resolves and loads dependencies (use this)
# insmod:   load exact file, no dependency resolution (rarely used)

# Unload modules
sudo modprobe -r tcp_bbr       # Remove (handles dependents)
sudo rmmod tcp_bbr             # Direct remove (fails if in use)

# Load at boot (persistent)
echo "br_netfilter" | sudo tee /etc/modules-load.d/kubernetes.conf
echo "overlay" | sudo tee -a /etc/modules-load.d/kubernetes.conf

# Or add to /etc/modules:
echo "tcp_bbr" | sudo tee -a /etc/modules

# Module parameters (persistent)
```

```
cat /etc/modprobe.d/blacklist.conf
# blacklist nouveau      blacklist NVIDIA nouveau driver
# blacklist pcspkr       disable PC speaker

# Check if module is loaded
lsmod | grep overlay
[ -d /sys/module/overlay ] && echo "loaded" || echo "not loaded"

# Show dependencies
modprobe --show-depends overlay

# Kubernetes pre-requisites (classic interview scenario):
cat /etc/modules-load.d/k8s.conf
# overlay
# br_netfilter

sudo modprobe overlay && sudo modprobe br_netfilter
lsmod | grep -E "overlay|br_netfilter"

# Required sysctl for K8s networking:
cat /etc/sysctl.d/k8s.conf
# net.bridge.bridge-nf-call-iptables = 1
# net.bridge.bridge-nf-call-ip6tables = 1
# net.ipv4.ip_forward = 1
sudo sysctl --system
```

🔷 Short crisp interview answer

> "Kernel modules are loadable extensions (.ko files) that add hardware support, filesystems, and protocols without rebooting. `lsmod` lists loaded modules. `modprobe` is the smart loader — it resolves dependencies automatically. `modprobe -r` unloads. For boot persistence, add module names to `/etc/modules-load.d/*.conf`. Classic production scenario: Kubernetes requires `br_netfilter` and `overlay` modules — I add them to modules-load.d and load immediately with modprobe."

1.13 Namespaces & Cgroups — Foundation of Containers

🔷 What namespaces are

```
Namespaces isolate global resources so each process sees its own view.

Linux has 8 namespace types:
┌──────────────────┬──────────────────────────────────────────────────┐
│ Namespace        │ What it isolates                                 │
├──────────────────┼──────────────────────────────────────────────────┤
```

```
│ PID            │ Process IDs (container sees its own PID 1)    │
│ Network (net)  │ Network interfaces, routes, iptables          │
│ Mount (mnt)    │ Filesystem mount points                       │
│ UTS            │ Hostname and domain name                      │
│ IPC            │ System V IPC, POSIX message queues            │
│ User           │ User/group IDs (container root != host root)  │
│ Cgroup         │ Cgroup root directory                         │
│ Time           │ System clocks (Linux 5.6+)                    │
└────────────────┴──────────────────────────────────────────────┘

A container = a process tree with all namespaces isolated from the host
```

🔷 Exploring namespaces

```
# View namespaces for a process — each is a symlink to a namespace ID
ls -la /proc/1/ns/
# lrwxrwxrwx net    -> net:[4026531992]
# lrwxrwxrwx pid    -> pid:[4026531836]
# lrwxrwxrwx mnt    -> mnt:[4026531840]

# Compare host process to container process
ls -la /proc/1/ns/net       # net:[4026531992]
ls -la /proc/12345/ns/net   # net:[4026532001] ← different namespace!

# Enter a container's namespace
sudo nsenter --target 12345 --net --pid -- ip addr
# Shows container's network interfaces from inside its namespace

# Create isolated environment
sudo unshare --pid --fork --mount-proc bash
# Inside: ps aux shows only this bash (PID 1 in this namespace)

# What Docker does:
# docker run = clone() with new namespaces for PID, net, mnt, UTS, IPC
# PID ns:  container processes start at PID 1
# net ns:  container gets its own network stack + eth0 (via veth pair)
# mnt ns:  container gets its own filesystem (overlayfs layers)
# UTS ns:  container has its own hostname

# List network namespaces
ip netns list
```

🔷 Cgroups — Resource control

```
# Cgroups limit and account for resource usage
# Every process belongs to a cgroup hierarchy
```

```
# cgroups v2 (modern, unified hierarchy):
# /sys/fs/cgroup/  — single unified tree

ls /sys/fs/cgroup/
# cgroup.controllers  cpu.stat  init.scope  memory.stat  system.slice

# View a process's cgroup
cat /proc/1234/cgroup
# 0::/system.slice/myapp.service   (cgroups v2)

# Per-service limits in systemd unit file:
[Service]
CPUQuota=50%           # Limit to 50% of one CPU
CPUWeight=100          # Relative CPU scheduling weight
MemoryLimit=512M       # Hard memory limit
MemoryHigh=400M        # Soft limit (throttle before hard)
IOWeight=50            # Relative I/O scheduling weight
TasksMax=256           # Max processes/threads

systemd-cgtop          # Real-time per-cgroup resource usage

# Docker resource limits → cgroup mappings
docker run --cpus=0.5 --memory=512m nginx
# Creates /sys/fs/cgroup/system.slice/docker-<id>.scope/
# cpu.max = 50000 100000  (50% of one CPU)
# memory.max = 536870912  (512MB)

# Kubernetes resources → cgroup mappings
# requests.cpu / limits.cpu  → cpu.shares / cpu.quota
# requests.memory / limits.memory → memory.request / memory.max

# OOM killer in cgroup context:
# dmesg: "oom-kill:constraint=CONSTRAINT_MEMCG" = cgroup OOM (container)
# dmesg: "oom-kill:constraint=CONSTRAINT_NONE" = system-wide OOM
```

🔷 Short crisp interview answer

> "Namespaces and cgroups are the two kernel primitives that make containers. Namespaces isolate — each container gets its own PID namespace (its own PID 1), network namespace (its own network stack), mount namespace (its own filesystem view), UTS namespace (its own hostname). Cgroups limit — they enforce CPU quotas, memory limits, I/O limits, and process count limits per container. Docker's `docker run` calls `clone()` with namespace flags, creates a cgroup hierarchy for resource limits, and sets up overlayfs for the layered filesystem. The container process IS a host process — just with an isolated view of the world and capped resources."

1.14 `udev` & Device Management — How Devices Appear in `/dev`

🔷 How udev works

```
Hardware appears (USB plugged in, disk detected)
        │
        ▼
Kernel detects device → creates uevent (broadcast to userspace)
        │
        ▼
udevd daemon receives uevent
        │
        ▼
udevd evaluates rules:
  /lib/udev/rules.d/*.rules   (package-provided)
  /etc/udev/rules.d/*.rules   (local overrides — processed last)
        │
        ▼
udevd creates device node in /dev/
Sets permissions and ownership
Creates symlinks (/dev/disk/by-uuid/, /dev/disk/by-id/, etc.)
Runs optional programs (load firmware, send notifications)
        │
        ▼
Device ready to use
```

🔷 Working with udev

```
# Monitor udev events live
udevadm monitor              # All events
udevadm monitor --kernel     # Kernel events only
udevadm monitor --udev       # Processed events only
# Plug in USB while watching — you'll see the event chain

# Inspect device attributes (for writing rules)
udevadm info /dev/sda
udevadm info --query=all --name=/dev/sda
# E: DEVNAME=/dev/sda
# E: ID_VENDOR=Samsung
# E: ID_MODEL=SSD_870_EVO
# E: ID_SERIAL=S59XNX0W123456
# E: MAJOR=8
# E: MINOR=0

udevadm info --attribute-walk --name=/dev/sda  # Walk parent devices

# Persistent device symlinks (created by udev rules)
ls -la /dev/disk/by-id/      # Persistent IDs (model + serial)
ls -la /dev/disk/by-uuid/    # UUIDs assigned by filesystem
ls -la /dev/disk/by-path/    # Physical path (PCI slot, port)
ls -la /dev/disk/by-label/   # Filesystem labels
```

```
# These always point to the right disk regardless of /dev/sdX naming
# This is why /etc/fstab uses UUIDs instead of /dev/sda!

# udev rules
# /etc/udev/rules.d/ — local rules (override /lib/udev/rules.d/)
# Files processed alphabetically; 99-local.rules wins over 10-usb.rules

# Rule syntax: MATCH_KEY=="value", ... ACTION_KEY="value"

# Give USB serial device consistent name:
# SUBSYSTEM=="tty", ATTRS{idVendor}=="0403", ATTRS{idProduct}=="6001",
#   SYMLINK+="arduino", MODE="0666"

# Set permissions on video device:
# KERNEL=="video[0-9]*", GROUP="video", MODE="0660"

# Run script when USB appears:
# ACTION=="add", SUBSYSTEM=="usb", ATTRS{idVendor}=="046d",
#   RUN+="/usr/local/bin/setup-usb.sh"

# Apply rules
sudo udevadm control --reload-rules    # Reload rules files
sudo udevadm trigger                   # Re-trigger for existing devices

# Test what rules apply to a device
udevadm test /sys/block/sda 2>&1 | grep -E "LINK|OWNER|GROUP|MODE"

# udevd is part of systemd on modern systems
systemctl status systemd-udevd
journalctl -u systemd-udevd
```

🔷 Short crisp interview answer

> "udev is the userspace device manager — when the kernel detects hardware it broadcasts a uevent; udevd receives it, evaluates rules from `/etc/udev/rules.d/` and `/lib/udev/rules.d/`, then creates the `/dev` node with correct permissions and symlinks. The symlinks in `/dev/disk/by-uuid/` are created by udev rules — that's why UUIDs in `/etc/fstab` always map to the right disk even if `/dev/sda` becomes `/dev/sdb` after hardware changes. I use `udevadm info /dev/sda` to see device attributes for writing rules, and `udevadm monitor` to watch events live when debugging device detection issues."

🏆 Category 1 — Complete Mental Model

```
LINUX SYSTEM LAYERS
═══════════════════
```

```
  Hardware
     │
     ▼
  Kernel (monolithic core + loadable modules)
     │ Exposes:
     ├── /proc/  ← process + kernel stats (virtual, in RAM)
     ├── /sys/   ← device/driver attributes (virtual, in RAM)
     ├── /dev/   ← device files (created by udev)
     └── System calls (user↔kernel API boundary)
     │
     ▼
  systemd (PID 1) — manages services, mounts, sockets, timers
     │
     ▼
  User Space
     ├── /etc/   ← configuration (static text)
     ├── /var/   ← variable data (logs, caches, state)
     ├── /usr/   ← programs, libraries, docs
     ├── /home/  ← user data
     └── /tmp/   ← temporary (cleared at boot)

BOOT SEQUENCE:
  BIOS/UEFI → GRUB → kernel + initramfs → systemd → services → ready

CONTAINER INTERNALS:
  Namespaces → isolate visibility (PID, net, mnt, UTS, IPC, user)
  Cgroups    → limit resources   (CPU, memory, I/O, processes)
  overlayfs  → layered filesystem (image layers + writable layer)
```

⚠ Master Gotcha List

#	Gotcha	Reality
1	`/etc/passwd` has passwords	Historic name — passwords are in `/etc/shadow`
2	`/proc` and `/sys` are real files	Virtual — in RAM only, nothing on disk
3	`usermod -G` adds to group	Replaces all groups — always use `usermod -aG`
4	`chmod -R 755 /var/www`	Sets files executable — use find+exec separately for files vs dirs
5	`rm -rf / tmp/` (space in path)	Destroys root filesystem! Always verify path before rm
6	`tail -f` after log rotation	Use `tail -F` (capital) — follows by filename, survives rotation
7	SUID on shell scripts works	Kernel ignores SUID on interpreted scripts — only ELF binaries

#	Gotcha	Reality
8	New group memberships take effect immediately	Require re-login (groups loaded at session start)
9	`2>&1 > file` redirects both to file	Backwards — stderr goes to terminal; correct: `> file 2>&1`
10	`systemctl start` persists across reboot	`start` is runtime only; `enable` makes it persist
11	`systemctl daemon-reload` is optional	Mandatory after editing any unit file — without it, old version used
12	Hard links work across filesystems	They can't — inodes are per-filesystem
13	`find -mtime +7` includes files from day 7	Strictly older than 7 days — day 7 is excluded
14	Container root = host root	User namespaces separate them; without user ns, they're the same UID 0

🔥 Top Interview Questions

Q1: What's the difference between `/proc` and `/sys`?

> Both are virtual filesystems existing only in RAM — the kernel generates content dynamically on read. `/proc` is older and less organized: it contains per-process directories (by PID) with cmdline, fds, memory maps, and limits, plus system-wide files like `/proc/meminfo` and `/proc/cpuinfo`. `/sys` (sysfs) is newer and more structured — one attribute per file, organized by device class and subsystem. `/proc/sys/` provides tunable kernel parameters, also accessible via `sysctl`. I use `/proc/PID/fd/` to see a process's open files, and `/sys/block/sda/queue/scheduler` to tune I/O schedulers.

Q2: A disk is full but `du` shows less than `df`. Why?

> Two common causes: (1) Deleted files held open by running processes — the directory entry is gone but blocks still exist because a process has an open file descriptor. Space is freed only when the last fd is closed. `lsof +L1` lists these "deleted but open" files. Fix: restart the process or truncate via its fd with `> /proc/<PID>/fd/<N>`. (2) Reserved blocks — ext4 reserves 5% for root by default; `df` counts these as used, `du` doesn't. Check with `tune2fs -l /dev/sda1 | grep Reserved`.

Q3: How do you give a process the ability to bind to port 80 without running as root?

> Three approaches: (1) Linux capabilities — `setcap cap_net_bind_service=ep /usr/local/bin/server` grants only the port-binding privilege. Run the service as an unprivileged user with this capability on the binary. (2) systemd socket activation — systemd binds the socket (as root) and passes the fd to the service; the service never binds itself. (3) Reverse proxy — nginx/HAProxy on port 80 proxying to the app on a high port. Modern best practice: capabilities via `setcap`.

Q4: What are the two kernel mechanisms that make containers possible?

> Namespaces and cgroups. Namespaces provide isolation: each container gets its own PID namespace (its own PID 1), network namespace (its own network stack with separate routing tables and iptables), mount namespace (its own filesystem view via overlayfs), and UTS namespace (its own hostname). Cgroups provide resource limits: CPU quotas, memory limits, I/O bandwidth limits, and process count limits per container. Docker's `docker run` calls `clone()` with namespace flags, creates a cgroup hierarchy, and sets up an overlayfs mount. The container is just a process with an isolated world-view and bounded resources.

Q5: What's the difference between a hard link and a soft link?

> A hard link is a second directory entry pointing to the same inode — both names are equally valid, the data survives deletion of either name, and link count decrements to 0 before data is freed. Hard links can't cross filesystem boundaries (inodes are per-filesystem) and can't link directories. A soft link is a file containing a path string — it can cross filesystems and link directories, but becomes dangling if the target is deleted. I use soft links for nginx `sites-enabled` (enable=create symlink, disable=remove it), Python version management, and cross-filesystem references.

Q6: Explain `umask` and how it interacts with file creation.

> `umask` is a bitmask subtracted from default permissions at creation time. Defaults are 666 for files and 777 for directories. With `umask 022`, files get 644 (rw-r--r--) and directories get 755 (rwxr-xr-x). For security-conscious environments, `umask 027` gives files 640 (group can read, others blocked) and directories 750. umask is per-process and inherited by child processes. For systemd services, set it with `UMask=0027` in the `[Service]` section — the shell's umask doesn't apply to services started by systemd.

Q7: Walk me through what happens when you type `systemctl restart nginx`.

> systemctl sends a D-Bus message to systemd (PID 1). systemd finds nginx's unit file, currently reads the `[Service]` section. It sends SIGTERM to nginx's main process (or the signal specified in `KillSignal=`), waits for `TimeoutStopSec` seconds for a clean exit, then SIGKILL if needed. Once the old process is gone, systemd forks, sets up the environment (user, cgroup, namespaces, file descriptor limits from the unit file), and execs the `ExecStart=` command. It then monitors the new process using the cgroup and reports status. If the service has `Type=notify`, systemd waits for an `sd_notify(READY=1)` signal before marking it active.

This document covers all 14 topics in Category 1: Linux Fundamentals — filesystem hierarchy, file operations, permissions, users, text tools, links, disk management, file descriptors, virtual filesystems, systemd, the boot process, kernel modules, namespaces/cgroups, and udev device management.